Building data course — Week 1 — find your way around (read-only)

sparql-playground / lessons

Generated for print and offline reading

Week 1 — find your way around (read-only)

Five daily lessons, all run from Python. Each computation is done twice, as a SQLite query string (stdlib `sqlite3`) and in pandas, and `lab.assert_parity` checks they agree. Run a day with `uv run python lessons/course.py run N` (it uses a temporary copy of `var/timeseries.sqlite`; build that with `uv run bldgts bootstrap`). `assert` lines state the answers you should see. Keep notes under the ignored `student-work/` folder.

Contents

- Day 1 — Open the database; what one row is
- Day 2 — SELECT, WHERE, ORDER BY, LIMIT on one sensor; what the UUIDs are
- Day 3 — Time: epoch seconds, `datetime()`, date ranges
- Day 4 — Summary stats: COUNT, MIN, MAX, AVG, SUM

- Day 5 — GROUP BY and HAVING: per day, per month, per series

---

Day 1 — Open the database; what one row is

Goal: open the API's database from Python and say in one sentence what a row means.

Read

An HVAC controller measures things (temperatures, pressures, fan status) and a **historian** saves a reading every few minutes. Building 50's historian exported a CSV with one column per point. `uv run bldgts bootstrap` turned that into the API's SQLite database, `var/timeseries.sqlite`.

SQLite is a whole SQL database in one file: no server, no login. Python ships with it (`import sqlite3`), and so do phones and browsers.

This course is driven from Python. Each lesson is a Markdown file whose ``` ``python blocks you can paste into a REPL or notebook, or run top to bottom withuv run python lessons/course.py run 1. That command works on a **temporary copy** of the database, so nothing you try can changevar/timeseries.sqlite`. (DB Browser for SQLite is still handy as a *viewer*: open the file read-only and browse. It is optional.)

Every computation is done **twice**, the way Open-FDD tests its rules:

- as **SQL**: a SQLite query held in a Python string, run through `lab.query`, which returns a DataFrame;

- as **pandas**: the same answer from a DataFrame, as an independent "oracle";
- then `lab.assert_parity(pandas_result, sql_result)` fails loudly if the two ever differ.

If you know pandas, SQL is the other way to say the same thing. If you know SQL, pandas is. Doing both is how you learn which one to reach for. A rough map for this week:

Question	SQL (runs inside the database)	pandas (runs on data already in memory)
which rows	`WHERE`	boolean mask, `df[mask]`
which columns	`SELECT a, b`	`df[["a", "b"]]`
sorted	`ORDER BY`	`sort_values`
first n	`LIMIT n`	`head(n)`
one number from many rows	`count`, `avg`, `sum` …	`.count()`, `.mean()`, `.sum()` …
one row per group	`GROUP BY`	`groupby(...).agg(...)`

The big difference is **where the data lives**. A SQL query is sent to the database and only the *answer* comes back. With pandas you first pull the rows into Python, then work on them. Much of this week is about when that matters.

The samples table

The first block opens the database and reads its structure from SQLite's own catalog, `sqlite_master`.

```
import time

import numpy as np
import pandas as pd

import lab

conn = lab.connect()  # sqlite3.connect() on the
        course database

tables = lab.query(
    conn,
    "SELECT name, sql FROM sqlite_master WHERE type
        = 'table' AND name NOT LIKE 'lab\\_%'
        ESCAPE '\\'",
)
print(tables.name.tolist())
print(tables.set_index("name").loc["samples",
        "sql"])
assert set(tables.name) == {"samples", "ingest_log"}
```

Table	What it holds
`samples`	every reading: `timeseries_id`, `ts`, `value`
`ingest_log`	one row per CSV the bootstrap loaded (file hash and row count), so it can skip unchanged files

The `CREATE TABLE` for `samples`:

```
CREATE TABLE samples (
    timeseries_id TEXT NOT NULL,
    ts INTEGER NOT NULL,
    value REAL,
    PRIMARY KEY (timeseries_id, ts)
) WITHOUT ROWID
```

Read it like this: - **One row is one reading:** this point (`timeseries_id`) had this `value` at this time (`ts`). - The **primary key** (`timeseries_id, ts`) means a point can't

have two readings at the same moment. - `WITHOUT ROWID` stores the rows *in a B-tree sorted by that key*, so all the readings of one point sit together, in time order. Day 2 and Day 12 show why that makes reads fast.

This shape is called **long** (or narrow) format: three columns, millions of rows. The CSV it came from was **wide**: one row per timestamp and one column per point. Day 11 converts between the two. A pandas DataFrame can be either; `SELECT * FROM samples` gives you a long one.

Run

How many readings are there? In SQL it's one aggregate. In pandas we have to load the table first.

```
COUNT_SQL = "SELECT count(*) AS n FROM samples"
t = time.perf_counter()
count_sql = lab.query(conn, COUNT_SQL)
t_count = time.perf_counter() - t
print(count_sql)

t = time.perf_counter()
samples = lab.query(
    conn, "SELECT timeseries_id, ts, value FROM
        samples"
)  # all rows, loaded ONCE this lesson
t_load = time.perf_counter() - t
count_pd = pd.DataFrame({"n": [len(samples)]})

lab.assert_parity(count_pd, count_sql, label="row
        count")
assert count_sql.n[0] == 4_193_248
```

Both say 4,193,248. Notice what it cost, though.

```
print(f"count(*) in SQL:        {t_count *
        1000:9.1f} ms   (returns 1 row)")
print(f"load into pandas first: {t_load * 1000:9.1f}
        ms   (returns {len(samples):,} rows)")
```

`count(*)` on a table stored as a B-tree walks the tree's leaf pages without building a single Python object. It is still O(n) in the worst case, but it moves **one number** into Python. pandas had to *materialise* 4.2 million rows first: also O(n), plus a copy of every row across the SQLite/Python boundary and a few hundred MB of memory. Pushing work into SQL means fewer bytes cross that boundary. This is the only time this lesson loads the whole table; most of the work this week loads one series (about 35,000 rows).

The first five rows, in key order.

```
FIRST5_SQL = "SELECT timeseries_id, ts, value FROM
        samples LIMIT 5"
first5_sql = lab.query(conn, FIRST5_SQL)
print(first5_sql)

# `samples` came back in primary-key order, so
        head() is the same five rows. (SQL promises
        no order without
# ORDER BY; it is key order here only because that
        is how the B-tree is laid out. Day 2 sorts
        explicitly.)
lab.assert_parity(samples.head(5), first5_sql,
        label="first five rows")
assert len(first5_sql) == 5
```

What did the bootstrap load?

```
LOG_SQL = "SELECT dataset_key, row_count FROM
        ingest_log ORDER BY dataset_key"
log_sql = lab.query(conn, LOG_SQL)
print(log_sql.to_string(index=False))
assert len(log_sql) == 2
assert log_sql.row_count.tolist() == [2_949_488,
        1_243_760]
assert log_sql.row_count.sum() == len(samples)
```

The two `row_counts` (2,949,488 for AHU_1 and 1,243,760 for AHU_2) add up to the total above.

Try it

1. Pick any `timeseries_id` from `samples` and look at its rows. Do all its rows share the same id and step forward in `ts`? (pandas: `samples[samples.timeseries_id == some_id]`; SQL: a `WHERE`, with the id passed as a parameter, not pasted into the string. Day 2 explains why.)
2. How many different points are there?

Answer

```
some_id = samples.timeseries_id.iloc[0]
one_pd = samples[samples.timeseries_id ==
         some_id].reset_index(
    drop=True
)  # O(n): looks at every row
one_sql = lab.query(
    conn,
    "SELECT timeseries_id, ts, value FROM samples
         WHERE timeseries_id = ? ORDER BY ts",  # B-
         tree seek
    (some_id,),
)
lab.assert_parity(one_pd, one_sql, keys=["ts"],
         label="one series")
assert one_sql.ts.is_monotonic_increasing and
         one_sql.ts.is_unique
print(one_sql.ts.diff().value_counts())  # every
         step is 300 s
assert one_sql.ts.diff().dropna().eq(300).all()

DISTINCT_SQL = "SELECT count(DISTINCT timeseries_id)
         AS series FROM samples"
distinct_sql = lab.query(conn, DISTINCT_SQL)
distinct_pd = pd.DataFrame({"series":
         [samples.timeseries_id.nunique()]})
lab.assert_parity(distinct_pd, distinct_sql,
         label="distinct series")
assert distinct_sql.series[0] == 118
assert len(samples) // 118 == 35_536
```

118 points (series). 4,193,248 / 118 = 35,536 readings each: every point was logged at every timestamp.

Why it matters

Every time-series database (Timescale, InfluxDB, a data lake) stores some version of *(series id, time, value)*. The hard part is never storing it. It's knowing **which** series id is the duct static pressure on AHU_1. That knowledge isn't in this table at all, and finding it is what the RDF half of the course is about.

Day 2 — SELECT, WHERE, ORDER BY, LIMIT on one sensor; what the UUIDs are

Goal: pull the readings of one sensor, sorted and trimmed, in SQL and in pandas, and know where its id comes from.

Read

A `timeseries_id` looks like `bf422925-1e2d-598d-9069-3b0f17d49c48`. That's a **UUID**, a 128-bit id written as 32 hex digits. These are version-5 UUIDs: a hash of a name, so the same building/point always gets the same UUID. They don't change between rebuilds, and the API can hand them out without exposing the BAS's internal point names.

The database doesn't know what a UUID means. That lives in the point list the model is built from, `model/points/BUILDING_50__AHU_1.csv`. Open it in a text editor or spreadsheet and find this row:

source_column	point_id	brick_class
outside_air_temp_f	AHU_1_OA_T	Outside_Air_Temperatu

So `bf422925…` is AHU_1's outside air temperature in °F. Here are a few you'll use this week:

Point	timeseries_id
AHU_1 outside air temp (°F)	`bf422925-1e2d-598d-9069-3b0f17d49c48`
AHU_1 supply fan status (1 = running)	`9ebf80dd-7821-510b-b2c0-cd0151e093c9`
AHU_1 duct static pressure (inH2O)	`52422584-9894-50bd-9696-2fbb8b062518`
AHU_1 duct static pressure setpoint (inH2O)	`761c2b61-4982-5157-bc66-2010cf6e2a47`

The parts of a query, in the order you write them, and the pandas that does each job:

```
SELECT  columns            -- what to show
df[["ts", "value"]]
FROM    table              -- where from
(the DataFrame you loaded)
WHERE   condition          -- which rows
df[df.value > 1.5]
ORDER BY column [DESC]     -- sorted how
.sort_values("value", ascending=False)
LIMIT   n                  -- how many
.head(n)
```

Text values go in single quotes in SQL. Double quotes are for column and table names.

Parameters, not f-strings. In Python you'll be tempted to build SQL with an f-string: `f"... WHERE timeseries_id = '{uid}'"`. Don't. If `uid` ever comes from a user, a file or an API request, a value like `x' OR '1'='1` rewrites your query (**SQL injection**), and even with trusted values you get quoting bugs. Instead leave a placeholder, `?` or a name like `:uid`, and pass the value separately. The database receives the SQL text and the value as two different things, so a value can never be

mistaken for SQL. It also lets SQLite reuse the compiled query. Every query in this course passes values as parameters.

Where the work happens. `WHERE timeseries_id = ?` is the best case for this table. The primary key is `(timeseries_id, ts)` and the rows are stored in that order in a B-tree, so SQLite **seeks** straight to the first row of that series and reads the next 35,536 rows in a row: O(log n + k) for n rows in the table and k rows returned. pandas has no such luxury. Once a table is loaded, `df[df.timeseries_id == uid]` compares every one of the n rows: O(n). (A pandas `set_index` on the id sorts the frame and lets it do the same kind of lookup, but you pay for that sort, and you had to load everything first.) So the usual pattern is: let SQL pick the rows, then use pandas on what comes back.

Run

```
import numpy as np
import pandas as pd

import lab

conn = lab.connect()

OAT = "bf422925-1e2d-598d-9069-3b0f17d49c48"  #
         AHU_1 outside air temp, F
FAN = "9ebf80dd-7821-510b-b2c0-cd0151e093c9"  #
         AHU_1 supply fan status, 0/1
STATIC = "52422584-9894-50bd-9696-2fbb8b062518"  #
         AHU_1 duct static pressure, inH2O
STATIC_SP = "761c2b61-4982-5157-bc66-2010cf6e2a47"
         # AHU_1 duct static pressure setpoint,
         inH2O

# Load ONE series for the pandas side. The id is a
         bound parameter.
SERIES_SQL = "SELECT ts, value FROM samples WHERE
         timeseries_id = :uid"


def series(uid):
    return lab.query(conn, SERIES_SQL, {"uid": uid})


oat = series(OAT)
print(len(oat), "rows")
assert len(oat) == 35_536
```

The first three outside air readings.

```
FIRST3_SQL = """
SELECT ts, value
FROM samples
WHERE timeseries_id = :uid
ORDER BY ts
LIMIT 3
"""
first3_sql = lab.query(conn, FIRST3_SQL, {"uid":
         OAT})
first3_pd =
         oat.sort_values("ts").head(3).reset_index(drop=True)
lab.assert_parity(first3_pd, first3_sql, keys=
         ["ts"], label="first 3 readings")
print(first3_sql)
assert first3_sql.iloc[0].tolist() == [1773621600,
         69.0327]
```

The hottest reading. Sort by value, biggest first, keep one row. Adding `ts` as a second sort key makes the answer deterministic if two readings tie.

```
HOTTEST_SQL = """
SELECT ts, value
FROM samples
WHERE timeseries_id = :uid
ORDER BY value DESC, ts
LIMIT 1
"""
hottest_sql = lab.query(conn, HOTTEST_SQL, {"uid":
         OAT})
hottest_pd = (
    oat.sort_values(["value", "ts"], ascending=
         [False,
         True]).head(1).reset_index(drop=True)
)
lab.assert_parity(hottest_pd, hottest_sql,
         label="hottest reading")
assert hottest_sql.iloc[0].tolist() == [1784065200,
         98.8608]
```

Readings above 1.5 inH2O on AHU_1's duct static pressure.

```
HIGH_SQL = """
SELECT count(*) AS n
FROM samples
WHERE timeseries_id = :uid
  AND value > :limit
"""
high_sql = lab.query(conn, HIGH_SQL, {"uid": STATIC,
         "limit": 1.5})
static = series(STATIC)
high_pd = pd.DataFrame(
    {"n": [int((static["value"] > 1.5).sum())]}
)  # boolean mask, then count the Trues
lab.assert_parity(high_pd, high_sql, label="static >
         1.5")
assert high_sql.n[0] == 8336
```

In pandas, `static["value"] > 1.5` makes a column of True/False, one per row, and `.sum()` counts the Trues. SQL's `WHERE` drops the rows and `count(*)` counts what's left. Same answer, same O(k) work over the series; the difference is that SQL never builds the 35,536-element mask or sends the rows to Python at all.

A look at the injection problem from above. The user-supplied string is treated as a plain id that matches nothing:

```
evil = "x' OR '1'='1"
evil_sql = lab.query(
    conn, "SELECT count(*) AS n FROM samples WHERE
         timeseries_id = :uid", {"uid": evil}
)
print(evil_sql.n[0], "rows for a hostile id")
assert evil_sql.n[0] == 0  # with an f-string this
         would have counted the whole table
```

The same slice through the API. A real application doesn't open the SQLite file. It asks a service over HTTP and gets JSON back. This repo has one (`bldgts serve`, a FastAPI app), and `lab.Api` drives it in-process with FastAPI's test client, so it runs offline against the same

database copy. The API speaks in **point ids** such as `AHU_1_OA_T`, names the building model knows; the model maps each point id to the `timeseries_id` UUID that the `samples` table is keyed on (Unit 4 asks the model that question in SPARQL). The service looks up the UUID, runs a SQL query much like the ones above, and returns the rows as JSON. Here is 1 July 2026 (288 readings) three ways. The API's `end` is inclusive, so we pass the last second of the day.

```
DAY_SQL = """
SELECT ts, value FROM samples
WHERE timeseries_id = :uid AND ts >= :start AND ts <
          :end
"""
t0 = 1782864000  # 2026-07-01 00:00 UTC
day_sql = lab.query(conn, DAY_SQL, {"uid": OAT,
          "start": t0, "end": t0 + 86_400})
day_pd = oat[(oat.ts >= t0) & (oat.ts < t0 +
          86_400)].reset_index(drop=True)

with lab.Api() as api:
    day_api = api.timeseries(
        "AHU_1_OA_T", start=t0, end=t0 + 86_399
    )  # HTTP GET -> JSON -> DataFrame
print(day_api.attrs)  # the model's point id -> UUID
          mapping, returned with the data

assert day_api.attrs["timeseries_id"] == OAT
lab.assert_parity(day_pd, day_sql, keys=["ts"],
          label="pandas == SQL")
lab.assert_parity(day_api, day_sql, keys=["ts"],
          label="API == SQL")
assert len(day_api) == 288
```

Three paths, one answer. The API adds a layer (HTTP, JSON, id lookup), which is how other programs, and people without database access, get the same data. It also caps the rows it returns, which is why we asked for a day, not all 35,536 readings.

`ts` is a big number. Tomorrow turns it into a date.

Try it

1. What was the coldest outside air reading?
2. Is AHU_1's supply fan status ever anything other than 0 or 1?

Answers

```
COLDEST_SQL = """
SELECT ts, value FROM samples
WHERE timeseries_id = :uid
ORDER BY value ASC, ts LIMIT 1
"""
coldest_sql = lab.query(conn, COLDEST_SQL, {"uid":
         OAT})
coldest_pd = oat.sort_values(["value",
         "ts"]).head(1).reset_index(drop=True)
lab.assert_parity(coldest_pd, coldest_sql,
         label="coldest reading")
assert coldest_sql.iloc[0].tolist() == [1773740400,
         17.8308]

DISTINCT_SQL = """
SELECT DISTINCT value FROM samples
WHERE timeseries_id = :uid
ORDER BY value
"""
distinct_sql = lab.query(conn, DISTINCT_SQL, {"uid":
         FAN})
distinct_pd = pd.DataFrame({"value":
         np.sort(series(FAN)["value"].unique())})
lab.assert_parity(distinct_pd, distinct_sql,
         label="fan status values")
assert distinct_sql.value.tolist() == [0.0, 1.0]
```

1. The coldest reading is 17.83 °F (it is stored as a decimal, `17.8308`) at `ts = 1773740400`.
2. Only 0 and 1. Status points are numbers in a historian, not booleans.

Why it matters

Pasting UUIDs by hand doesn't scale. AHU_1 alone has 83 points, and a campus has tens of thousands. Day 6 puts the point list in a table, and Unit 4 replaces it with a SPARQL query: "give me the duct static pressure sensor of every AHU" → UUIDs. And because those UUIDs will come from a query, not from your own keyboard, passing them as parameters is not just tidy. It is how you keep a bad id from becoming a bad query.

Day 3 — Time: epoch seconds, `datetime()`, date ranges

Goal: turn `ts` into readable dates, and select a day or a month, in SQL and in pandas.

Read

`ts` is **Unix epoch seconds**: seconds since 1970-01-01 00:00:00 UTC. `1773621600` is 2026-03-16 00:40 UTC. Historians store time like this because integers sort fast, compare exactly and have no time zone ambiguity. Everything in this database is **UTC**.

SQLite's date functions read epoch seconds when you add the `'unixepoch'` modifier. pandas does it with `pd.to_datetime(..., unit="s", utc=True)`. Here is how the two line up for `1773621600`:

SQL	pandas (`dt = pd.to_datetime(ts, unit="s", utc=True)`)	Result
`datetime(ts, 'unixepoch')`	`dt.dt.strftime("%Y-%m-%d %H:%M:%S")`	`2026-03-16 00:40:00`
`date(ts, 'unixepoch')`	`dt.dt.strftime("%Y-%m-%d")`	`2026-03-16`
`strftime('%H', ts, 'unixepoch')`	`dt.dt.strftime("%H")` or `dt.dt.hour`	`00` (hour)
`strftime('%w', ts, 'unixepoch')`	`(dt.dt.dayofweek + 1) % 7`	`1` (day of week, **0 = Sunday** in SQL, 0 = Monday in pandas)

SQL	pandas (`dt = pd.to_datetime(ts, unit="s", utc=True)`)	Result
`unixepoch('2026-07-01')`	`int(pd.Timestamp("2026-07-01", tz="UTC").timestamp())`	`1782864000` (text → epoch seconds)

In pandas you get a real datetime column with the whole `.dt` toolbox (`.dt.date`, `.dt.floor("D")`, `.dt.tz_convert(...)`). SQLite has no date type at all: a "date" there is text (or a number) and the date functions compute on it row by row.

To select a time range, compare `ts` to epoch numbers. Write the range **half-open**, `start <= ts < end`, so consecutive days never share a reading.

Don't filter with `date(ts, 'unixepoch') = '2026-07-01'`. It gives the same rows, but SQLite has to compute a date for every row instead of jumping to the range. The pandas twin of that mistake is converting a whole column to datetimes just to filter it; comparing the integer column to two numbers is a plain vectorized mask.

Why does the first form let SQLite jump? The primary key is `(timeseries_id, ts)`, so within one series the rows are sorted by `ts`. `ts >= a AND ts < b` is a **range seek**: find `a` by walking the B-tree from the root, O(log n), then read the k rows until `b`. Wrapping `ts` in a function hides it from the index, so SQLite falls back to reading all of the series' rows and testing each. `EXPLAIN QUERY PLAN` shows which one it picked (below).

Time zones. Store UTC, convert at the edges. pandas can convert to a named zone (`America/Chicago`) and handle daylight saving. SQLite can only do `'localtime'`,

the *machine's* zone, so a query's answer would change with whoever runs it. That's another reason to keep zone conversion in pandas or application code.

Run

```
import pandas as pd

import lab

conn = lab.connect()

OAT = "bf422925-1e2d-598d-9069-3b0f17d49c48"  #
        AHU_1 outside air temp, F
oat = lab.query(conn, "SELECT ts, value FROM samples
        WHERE timeseries_id = :uid", {"uid": OAT})
oat["time_utc"] = pd.to_datetime(oat["ts"],
        unit="s", utc=True)  # tz-aware
        datetime64[ns, UTC]
print(oat.dtypes)
```

The first three readings as dates. SQL's `datetime()` returns text, so we format the pandas side the same way.

```
FIRST3_SQL = """
SELECT datetime(ts, 'unixepoch') AS time_utc, value
FROM samples
WHERE timeseries_id = :uid
ORDER BY ts
LIMIT 3
"""
first3_sql = lab.query(conn, FIRST3_SQL, {"uid":
        OAT})
first3_pd = (
    oat.sort_values("ts")
    .head(3)
    .assign(time_utc=lambda d:
        d["time_utc"].dt.strftime("%Y-%m-%d
        %H:%M:%S"))[
        ["time_utc", "value"]
    ]
    .reset_index(drop=True)
)
lab.assert_parity(first3_pd, first3_sql, keys=
        ["time_utc"], label="first 3 as dates")
print(first3_sql)
assert first3_sql.iloc[0].tolist() == ["2026-03-16
        00:40:00", 69.0327]
```

First and last timestamp in the whole database. Every series covers the same period (Day 5 checks), so one series is a fair oracle for the whole table. SQL answers over all 4.2 million rows without loading any.

```
RANGE_SQL = """
SELECT datetime(min(ts), 'unixepoch') AS first_utc,
         datetime(max(ts), 'unixepoch') AS last_utc
         FROM samples
"""
range_sql = lab.query(conn, RANGE_SQL)
fmt = "%Y-%m-%d %H:%M:%S"
range_pd = pd.DataFrame(
    {
        "first_utc":
         [oat.time_utc.min().strftime(fmt)],
        "last_utc":
         [oat.time_utc.max().strftime(fmt)],
    }
)
lab.assert_parity(range_pd, range_sql, label="first
         and last timestamp")
print(range_sql.iloc[0].tolist())
assert range_sql.iloc[0].tolist() == ["2026-03-16
         00:40:00", "2026-07-17 09:55:00"]
```

All outside air readings on 1 July 2026 (UTC).
Note the epoch bounds are computed once, by the database in SQL and by pandas in Python, and compared as plain numbers.

```
DAY_SQL = """
SELECT count(*) AS n
FROM samples
WHERE timeseries_id = :uid
  AND ts >= unixepoch(:start)
  AND ts <  unixepoch(:end)
"""
day_sql = lab.query(conn, DAY_SQL, {"uid": OAT,
          "start": "2026-07-01", "end": "2026-07-
          02"})

start = int(pd.Timestamp("2026-07-01",
          tz="UTC").timestamp())
end = int(pd.Timestamp("2026-07-02",
          tz="UTC").timestamp())
assert start == 1782864000
day_pd = pd.DataFrame({"n": [int(((oat.ts >= start)
          & (oat.ts < end)).sum())]})
lab.assert_parity(day_pd, day_sql, label="readings
          on 2026-07-01")
assert day_sql.n[0] == 288
```

288 = 24 hours × 12 readings an hour: a full day of 5-minute data.

Mixing naive and time-zone-aware datetimes is the classic pandas time bug, so pandas refuses to compare them:

```
naive = pd.to_datetime(oat["ts"], unit="s")  # no
          utc=True: naive datetimes
try:
    naive >= pd.Timestamp("2026-07-01", tz="UTC")
except TypeError as err:
    print("TypeError:", err)
else:
    raise AssertionError("expected a naive/aware
          comparison to fail")
```

That error is the friendly outcome. Always pass `utc=True` when you convert epoch seconds, and build range bounds with `tz="UTC"`.

Which plan does SQLite pick? `EXPLAIN QUERY PLAN` shows whether the range became an index seek.

```
RANGE_FILTER = "SELECT count(*) FROM samples WHERE
        timeseries_id = ? AND ts >= ? AND ts < ?"
FUNCTION_FILTER = (
    "SELECT count(*) FROM samples WHERE
        timeseries_id = ? AND date(ts, 'unixepoch')
        = ?"
)


def plan(sql, params):
    return conn.execute("EXPLAIN QUERY PLAN " + sql,
        params).fetchall()[0][-1]


p_range = plan(RANGE_FILTER, (OAT, start, end))
p_func = plan(FUNCTION_FILTER, (OAT, "2026-07-01"))
print(p_range)
print(p_func)
assert "ts>" in p_range and "ts>" not in p_func  #
        the range is part of the seek, or it isn't

same_rows = conn.execute(FUNCTION_FILTER, (OAT,
        "2026-07-01")).fetchone()[0]
assert same_rows == day_sql.n[0]  # identical
        answer, different amount of work

t_range = lab.best_time(lambda:
        conn.execute(RANGE_FILTER, (OAT, start,
        end)).fetchone())
t_func = lab.best_time(lambda:
        conn.execute(FUNCTION_FILTER, (OAT, "2026-
        07-01")).fetchone())
print(f"range seek: {t_range * 1e6:8.0f} us   date()
        on every row: {t_func * 1e6:8.0f} us")
```

Same answer both ways, but the first reads about 288 rows and the second reads all 35,536 of the series and runs `date()` on each. Across a whole table that gap is the difference between milliseconds and many seconds.

Try it

1. When was the hottest outside air reading, as a date and time?
2. How many readings does the first day, 2026-03-16, have, and why isn't it 288?
3. Show that hottest reading in `America/Chicago` time. (pandas only: SQLite has no named zones.)

Answers

```
HOT_SQL = """
SELECT datetime(ts, 'unixepoch') AS time_utc, value
         FROM samples
WHERE timeseries_id = :uid
ORDER BY value DESC, ts LIMIT 1
"""
hot_sql = lab.query(conn, HOT_SQL, {"uid": OAT})
hot_pd = (
    oat.sort_values(["value", "ts"], ascending=
         [False, True])
    .head(1)
    .assign(time_utc=lambda d:
         d["time_utc"].dt.strftime(fmt))
         [["time_utc", "value"]]
    .reset_index(drop=True)
)
lab.assert_parity(hot_pd, hot_sql, label="hottest
         reading")
assert hot_sql.iloc[0].tolist() == ["2026-07-14
         21:40:00", 98.8608]

FIRST_DAY_SQL = """
SELECT count(*) AS n FROM samples
WHERE timeseries_id = :uid AND ts >=
         unixepoch(:start) AND ts < unixepoch(:end)
"""
first_day_sql = lab.query(
    conn, FIRST_DAY_SQL, {"uid": OAT, "start":
         "2026-03-16", "end": "2026-03-17"}
)
lo = int(pd.Timestamp("2026-03-16",
         tz="UTC").timestamp())
first_day_pd = pd.DataFrame({"n": [int(((oat.ts >=
         lo) & (oat.ts < lo + 86_400)).sum())]})
lab.assert_parity(first_day_pd, first_day_sql,
         label="readings on 2026-03-16")
assert first_day_sql.n[0] == 280

hot_local = oat.loc[oat.value.idxmax(),
         "time_utc"].tz_convert("America/Chicago")
print(hot_local)  # 2026-07-14 16:40:00-05:00
assert hot_local.hour == 16  # CDT = UTC-5
```

1. 2026-07-14 21:40:00 UTC, 98.86 °F.

2280. The data starts at 00:40, not midnight, so the first eight readings of that day were never logged.
3. 16:40 local (UTC-5 in July): 21:40 UTC is mid-afternoon, which is when you'd expect the hottest reading. Remember that the database is UTC and the *building* is not, which is why "6 am" shows up elsewhere.

Why it matters

Building people think in local time ("the fan should start at 6 am"). Databases should store UTC. Converting at the edges, and noticing when a "6 am start" shows up at 11:00 UTC, is a daily skill. The RDF model will record the site's time zone so code doesn't have to guess.

Day 4 — Summary stats: COUNT, MIN, MAX, AVG, SUM

Goal: summarize a sensor in one row, in SQL and in pandas, and turn a status point into run hours.

Read

Aggregate functions fold many rows into one value. pandas has a method for each:

SQL	pandas (on a column `v`)	Notes
`count(*)`	`len(df)`	rows
`count(value)`	`v.count()`	rows where `value` is not NULL / NaN
`min(value)`, `max(value)`	`v.min()`, `v.max()`	ignore NULLs / NaN
`avg(value)`	`v.mean()`	mean of the non-NULL values
`sum(value)`	`v.sum()`	total (SQL: NULL for no rows; pandas: 0)
`round(x, n)`	`round(x, n)`	for reading, not storage. **Never round before comparing**

Without `GROUP BY` (tomorrow), an aggregate query returns exactly one row.

A useful trick: a status point is 1 when running and 0 when off. With one reading every 5 minutes, `sum(status) × 5 / 60` is the **run hours**. Use `60.0`, not `60`: integer

division in SQL drops the remainder. (In Python 3, `/` is always true division, so pandas has no such trap. Different engines, different gotchas.)

How each engine does it. Both are one pass over the rows, O(k) for k rows. SQLite computes every aggregate in the `SELECT` list in the *same* pass, while it streams the rows off the B-tree, and returns a single row. pandas works on a column in memory: each of `min`, `max`, `mean` is its own fast, vectorized (compiled C loop) pass, and the rows had to be loaded into Python first. So SQL moves one row of results; pandas moves k rows of input. Pushing the aggregate into SQL is cheaper whenever you only need the summary, as the timing below shows.

Run

```
import sqlite3

import numpy as np
import pandas as pd

import lab

conn = lab.connect()

OAT = "bf422925-1e2d-598d-9069-3b0f17d49c48"  #
         AHU_1 outside air temp, F
FAN = "9ebf80dd-7821-510b-b2c0-cd0151e093c9"  #
         AHU_1 supply fan status, 0/1
STATIC = "52422584-9894-50bd-9696-2fbb8b062518"  #
         AHU_1 duct static pressure, inH2O
STATIC_SP = "761c2b61-4982-5157-bc66-2010cf6e2a47"
         # AHU_1 duct static pressure setpoint,
         inH2O

SERIES_SQL = "SELECT ts, value FROM samples WHERE
         timeseries_id = :uid"


def series(uid):
    return lab.query(conn, SERIES_SQL, {"uid": uid})


oat, fan, static, static_sp = (series(u) for u in
         (OAT, FAN, STATIC, STATIC_SP))
```

AHU_1 outside air temperature, whole dataset.
SQL computes the four statistics unrounded, in one pass.

```
SUMMARY_SQL = """
SELECT count(*)   AS n,
       min(value) AS min_f,
       max(value) AS max_f,
       avg(value) AS avg_f
FROM samples
WHERE timeseries_id = :uid
"""
summary_sql = lab.query(conn, SUMMARY_SQL, {"uid":
        OAT})
v = oat["value"]
summary_pd = pd.DataFrame(
    {"n": [len(oat)], "min_f": [v.min()], "max_f":
        [v.max()], "avg_f": [v.mean()]}
)
lab.assert_parity(
    summary_pd, summary_sql, label="OAT summary"
)  # unrounded: compare the real numbers

shown = summary_sql.iloc[0]
print(
    summary_sql.round({"min_f": 2, "max_f": 2,
        "avg_f": 3}).to_dict("records")[0]
)  # round only to print
assert (int(shown.n), round(shown.min_f, 2),
        round(shown.max_f, 2), round(shown.avg_f,
        3)) == (
    35536,
    17.83,
    98.86,
    61.276,
)
```

The same for one day. Add a half-open range in `ts` to the `WHERE` (Day 3); in pandas it's a mask first.

```
DAY_SQL = """
SELECT min(value) AS min_f, max(value) AS max_f,
         avg(value) AS avg_f
FROM samples
WHERE timeseries_id = :uid
  AND ts >= unixepoch(:start) AND ts <
         unixepoch(:end)
"""
day_sql = lab.query(conn, DAY_SQL, {"uid": OAT,
         "start": "2026-07-01", "end": "2026-07-
         02"})
lo = int(pd.Timestamp("2026-07-01",
         tz="UTC").timestamp())
day = oat[(oat.ts >= lo) & (oat.ts < lo + 86_400)]
         ["value"]
day_pd = pd.DataFrame({"min_f": [day.min()],
         "max_f": [day.max()], "avg_f":
         [day.mean()]})
lab.assert_parity(day_pd, day_sql, label="OAT on
         2026-07-01")
assert day_sql.iloc[0].round(1).tolist() == [79.9,
         97.1, 88.3]
```

AHU_1 supply fan run hours. The status is 0/1, so its sum is the number of readings with the fan on.

```
HOURS_SQL = "SELECT sum(value) * 5 / 60.0 AS
         run_hours FROM samples WHERE timeseries_id
         = :uid"
hours_sql = lab.query(conn, HOURS_SQL, {"uid": FAN})
hours_pd = pd.DataFrame({"run_hours":
         [fan["value"].sum() * 5 / 60]})
lab.assert_parity(hours_pd, hours_sql, label="fan
         run hours")
print(hours_sql.run_hours[0])
assert round(hours_sql.run_hours[0], 1) == 1574.4
```

The dataset covers about 2,961 hours (35,536 readings × 5 minutes), so the fan ran a bit over half the time.

NULLs and integer division. The real series have no NULLs, so here is a three-row table in a throwaway in-memory database, to see both engines skip them the same way and to see the `60` vs `60.0` trap.

```
mem = sqlite3.connect(":memory:")
mem.execute("CREATE TABLE t (value REAL)")
mem.executemany("INSERT INTO t VALUES (?)", [(1.0,),
        (None,), (3.0,)])
null_sql = pd.read_sql_query(
    "SELECT count(*) AS n_rows, count(value) AS
        n_values, avg(value) AS mean FROM t", mem
)
t = pd.Series([1.0, np.nan, 3.0])
null_pd = pd.DataFrame({"n_rows": [len(t)],
        "n_values": [t.count()], "mean":
        [t.mean()]})
lab.assert_parity(
    null_pd, null_sql, label="NULL / NaN handling"
)  # 3 rows, 2 values, mean 2.0 not 4/3

print(mem.execute("SELECT 7 * 5 / 60, 7 * 5 /
        60.0").fetchone())  # (0, 0.5833...)
assert mem.execute("SELECT 7 * 5 / 60").fetchone()
        [0] == 0
```

Where the time goes. The summary you want is a single number. Compare asking SQL for it with loading the series into pandas first and calling `.mean()`:

```
AVG_SQL = "SELECT avg(value) FROM samples WHERE
        timeseries_id = ?"
t_sql = lab.best_time(lambda: conn.execute(AVG_SQL,
        (STATIC,)).fetchone()[0])
t_load_mean = lab.best_time(lambda: lab.query(conn,
        SERIES_SQL, {"uid": STATIC})
        ["value"].mean())
t_mean_only = lab.best_time(lambda:
        static["value"].mean())  # series already
        in memory
print(f"SQL avg(), one row back:         {t_sql *
        1000:7.2f} ms")
print(f"load series, then .mean():
        {t_load_mean * 1000:7.2f} ms")
print(f"  .mean() alone (already loaded):
        {t_mean_only * 1000:7.2f} ms")
assert abs(conn.execute(AVG_SQL,
        (STATIC,)).fetchone()[0] -
        static["value"].mean()) < 1e-9
```

The `.mean()` itself is a few tenths of a millisecond; vectorized pandas is fast. Nearly all of the pandas cost is **getting the rows into Python**. If you will ask many questions of the same series, load it once (as this lesson does) and the load amortises. If you only need one summary of a large table, let SQL do it and move one row.

Try it

1. What are the average duct static pressure and its average setpoint on AHU_1? Which is bigger?
2. What fraction of the time was AHU_1's fan running? (Hint: the average of a 0/1 column is the fraction.)

Answers

```
AVG_ONE_SQL = "SELECT avg(value) AS avg_value FROM
          samples WHERE timeseries_id = :uid"
for name, uid, frame, want in [
    ("static", STATIC, static, 0.783),
    ("static setpoint", STATIC_SP, static_sp,
          1.366),
    ("fan running fraction", FAN, fan, 0.532),
]:
    sql = lab.query(conn, AVG_ONE_SQL, {"uid": uid})
    pdf = pd.DataFrame({"avg_value":
          [frame["value"].mean()]})
    lab.assert_parity(pdf, sql, label=name)
    assert round(sql.avg_value[0], 3) == want,
          (name, sql.avg_value[0])
    print(f"{name}: {sql.avg_value[0]:.3f}")
```

The average static (0.78) is far below the average setpoint (1.37). But that includes all the hours the fan was off, when static is near zero. Day 8 asks the fair question: what is static *while the fan runs*. The fan ran 53.2% of the time, which matches the 1,574 run hours out of about 2,961.

Why it matters

Averages over everything hide what matters. A sensor reading 0 °F because the unit is off pulls the average down just like a cold day does. Most of fault detection is asking “the average **of what, when**?” That’s filtering (Days 8–9) and gating (week 3).

Day 5 — GROUP BY and HAVING: per day, per month, per series

Goal: one summary row per group, and keep only the groups you care about, in SQL and in pandas.

Read

`GROUP BY` splits rows into groups. Each aggregate is then computed once per group:

```
SELECT group_columns, aggregates
FROM ...
WHERE   row filter           -- before grouping
GROUP BY group_columns
HAVING  group filter          -- after grouping, can
use aggregates
ORDER BY ...
```

`WHERE` throws away rows before they're grouped. `HAVING` throws away whole groups after. You can't put `avg(...)` in a `WHERE`, because no group exists yet at that point.

You can group by an expression, such as the date, so `GROUP BY date(ts, 'unixepoch')` gives one row per day. SQLite lets you use a column alias (`AS day`) in `GROUP BY` and `ORDER BY`.

The pandas version of each clause:

SQL	pandas
`WHERE cond`	`df[mask]` before the `groupby`
`GROUP BY key`	`df.groupby(key)` (key can be a column or a computed Series, like a formatted date)

SQL	pandas
`avg(value), count(*), sum(value)`	`.agg(avg_f=("value", "mean"), n=("value", "size"), ...)`
`HAVING count(*) < 288`	filter **after** the groupby: `grouped[grouped.n < 288]`
`ORDER BY x DESC LIMIT 1`	`.sort_values("x", ascending=False).head(1)`

So `HAVING` is not a special pandas feature. It's just an ordinary filter on the small table `groupby` gives you. `WHERE` and `HAVING` differ in SQL only because of clause order; in pandas the order of your statements says it.

How each engine groups. pandas **hashes**: one pass over the rows, each row dropped into a bucket for its key, O(n). SQLite either **sorts** the rows by the key (into a temporary B-tree) and sums runs of equal keys, which is O(n log n), or, if the rows already come out in key order, skips the sort and streams. The primary key is `(timeseries_id, ts)`, so grouping by `timeseries_id` needs no sorting, but grouping by an expression of `ts`, like the day, does. `EXPLAIN QUERY PLAN` says which (below).

Run

```
import time

import pandas as pd

import lab

conn = lab.connect()

OAT = "bf422925-1e2d-598d-9069-3b0f17d49c48"  #
          AHU_1 outside air temp, F
FAN = "9ebf80dd-7821-510b-b2c0-cd0151e093c9"  #
          AHU_1 supply fan status, 0/1

SERIES_SQL = "SELECT ts, value FROM samples WHERE
          timeseries_id = :uid"


def series(uid):
    df = lab.query(conn, SERIES_SQL, {"uid": uid})
    dt = pd.to_datetime(df["ts"], unit="s",
          utc=True)
    return df.assign(day=dt.dt.strftime("%Y-%m-%d"),
          month=dt.dt.strftime("%Y-%m"))


oat, fan = (
    series(OAT),
    series(FAN),
)  # one series each; day/month are the pandas twins
          of date()/strftime()
```

Monthly average outside air temperature. `groupby` on a column, `agg` for the statistic.

```
MONTHLY_SQL = """
SELECT strftime('%Y-%m', ts, 'unixepoch') AS month,
       avg(value) AS avg_f
FROM samples
WHERE timeseries_id = :uid
GROUP BY month
ORDER BY month
"""
monthly_sql = lab.query(conn, MONTHLY_SQL, {"uid":
         OAT})
monthly_pd = oat.groupby("month").agg(avg_f=
         ("value", "mean")).reset_index()
lab.assert_parity(monthly_pd, monthly_sql, keys=
         ["month"], label="monthly average")
print(monthly_sql.round(1).to_string(index=False))
assert len(monthly_sql) == 5  # March to July
assert monthly_sql.iloc[0].month == "2026-03" and
         round(monthly_sql.iloc[0].avg_f, 1) == 41.5
```

The hottest day on average. Group by day, then sort the small result and keep the top row.

```
HOTDAY_SQL = """
SELECT date(ts, 'unixepoch') AS day, avg(value) AS
         avg_f
FROM samples
WHERE timeseries_id = :uid
GROUP BY day
ORDER BY avg_f DESC
LIMIT 1
"""
hotday_sql = lab.query(conn, HOTDAY_SQL, {"uid":
         OAT})
daily_avg = oat.groupby("day").agg(avg_f=("value",
         "mean")).reset_index()
hotday_pd = daily_avg.sort_values("avg_f",
         ascending=False).head(1).reset_index(drop=True)
lab.assert_parity(hotday_pd, hotday_sql,
         label="hottest day")
assert hotday_sql.iloc[0].day == "2026-07-01" and
         round(hotday_sql.iloc[0].avg_f, 1) == 88.3
```

1 July (88.30) edges out 2 July (88.25) by 0.04 °F, so the two look tied at one decimal. The sort used the unrounded averages. Round for display only, as in Day 4.

Days that don't have a full 288 readings. The condition is on the group's `count(*)`, so it must be `HAVING`.

```
SHORT_SQL = """
SELECT date(ts, 'unixepoch') AS day, count(*) AS n
FROM samples
WHERE timeseries_id = :uid
GROUP BY day
HAVING count(*) < :full_day
ORDER BY day
"""
short_sql = lab.query(conn, SHORT_SQL, {"uid": OAT,
          "full_day": 288})
counts = oat.groupby("day").agg(n=("value",
          "size")).reset_index()
short_pd = counts[counts.n < 288].reset_index(
    drop=True
)  # HAVING = an ordinary filter after the groupby
lab.assert_parity(short_pd, short_sql, keys=["day"],
          label="short days")
print(short_sql.to_string(index=False))
assert short_sql.day.tolist() == ["2026-03-16",
          "2026-07-17"]
assert short_sql.n.tolist() == [280, 120]
```

The two short days are the first (2026-03-16, starts at 00:40, 280 readings) and the last (2026-07-17, ends at 09:55, 120 readings). There are no gaps in between. Real historians usually aren't this tidy, so checking is always worth it.

`WHERE` vs `HAVING`. A `WHERE` on `value` filters rows *before* counting, so it changes what is counted. Count only the readings above 90 °F per day, and the days with no such readings vanish, because no rows reach the group:

```
HOT_COUNT_SQL = """
SELECT date(ts, 'unixepoch') AS day, count(*) AS
        n_hot
FROM samples
WHERE timeseries_id = :uid AND value > 90      --
        row filter, before grouping
GROUP BY day
ORDER BY day
"""
hot_sql = lab.query(conn, HOT_COUNT_SQL, {"uid":
        OAT})
hot_pd = (
    oat[oat.value > 90].groupby("day").agg(n_hot=
        ("value", "size")).reset_index()
)  # mask, then group
lab.assert_parity(hot_pd, hot_sql, keys=["day"],
        label="WHERE before GROUP BY")
print(len(hot_sql), "of", counts.shape[0], "days
        ever passed 90 F")
assert len(hot_sql) < len(counts)
```

Per series. Do all 118 series have the same number of readings? This is the one place that needs the whole table in pandas, so we load just the id column, once.

```
PER_SERIES_SQL = "SELECT timeseries_id, count(*) AS
          n FROM samples GROUP BY timeseries_id"

t = time.perf_counter()
per_series_sql = lab.query(conn, PER_SERIES_SQL)
t_sql = time.perf_counter() - t

t = time.perf_counter()
ids = lab.query(conn, "SELECT timeseries_id FROM
          samples")  # 4.2 million ids, loaded once
per_series_pd = ids.groupby("timeseries_id").agg(n=
          ("timeseries_id", "size")).reset_index()
t_pd = time.perf_counter() - t

lab.assert_parity(
    per_series_pd, per_series_sql, keys=
          ["timeseries_id"], label="readings per
          series"
)
print(
    f"SQL GROUP BY: {t_sql * 1000:7.0f} ms (118 rows
          back)   load + groupby: {t_pd * 1000:7.0f}
          ms"
)
assert (
    per_series_sql.n.nunique() == 1 and
          per_series_sql.n[0] == 35_536 and
          len(per_series_sql) == 118
)
del ids  # free the 4.2 million rows
```

One distinct count for everyone: all 118 series have 35,536 readings. The SQL grouped in a single streaming pass without a sort; pandas first had to pull 4.2 million strings into Python. Same answer, much less data moved.

How SQLite plans each `GROUP BY`.

```
def plan(sql):
    return " | ".join(r[-1] for r in
          conn.execute("EXPLAIN QUERY PLAN " + sql,
          {"uid": OAT}))


p_series = plan(PER_SERIES_SQL)
p_day = plan(HOTDAY_SQL)
print(p_series)
print(p_day)
assert "TEMP B-TREE" not in p_series  # already in
          key order: stream, no sort
assert "TEMP B-TREE" in p_day  # grouping by a
          function of ts needs a temporary sorted
          structure
```

Try it

1. Readings per series: does every one of the 118 series have the same count? (The per-series block above did it with `GROUP BY`. Now count how many series share each count, in SQL with a **subquery**: a query in a `FROM`.)
2. Daily run hours of AHU_1's supply fan. Which day ran longest?

Answers

```
SAME_COUNT_SQL = """
SELECT n AS readings, count(*) AS series
FROM (SELECT timeseries_id, count(*) AS n FROM
         samples GROUP BY timeseries_id)
GROUP BY n
"""
same_sql = lab.query(conn, SAME_COUNT_SQL)
same_pd = (
    per_series_pd.groupby("n")
    .size()
    .rename("series")
    .reset_index()
    .rename(columns={"n": "readings"})
)
lab.assert_parity(same_pd, same_sql, keys=
         ["readings"], label="series per count")
assert same_sql.iloc[0].tolist() == [35_536, 118]
assert len(same_sql) == 1

RUN_HOURS_SQL = """
SELECT date(ts, 'unixepoch') AS day, sum(value) * 5
         / 60.0 AS run_hours
FROM samples
WHERE timeseries_id = :uid
GROUP BY day
ORDER BY run_hours DESC, day
LIMIT 3
"""
run_sql = lab.query(conn, RUN_HOURS_SQL, {"uid":
         FAN})
daily_hours = fan.groupby("day").agg(run_hours=
         ("value", lambda v: v.sum() * 5 /
         60)).reset_index()
run_pd = (
    daily_hours.sort_values(["run_hours", "day"],
         ascending=[False, True])
    .head(3)
    .reset_index(drop=True)
)
lab.assert_parity(run_pd, run_sql, keys=["day"],
         label="longest run days")
print(run_sql.round(1).to_string(index=False))
assert len(run_sql) == 3
assert run_sql.run_hours.iloc[0] == 24.0
```

A query in a `FROM` is a **subquery**: a table made on the fly. In pandas it is just the previous step's result. One row back means one count for everyone: all 118 series have 35,536 readings.

Several days ran all 24 hours. Is that normal for an office AHU? That's a question for the building's operators, and it's the kind of finding retro-commissioning starts from.

Week 1 challenge

Using only the four UUIDs from Day 2, write **one query** that reports, for each month: the average outside air temperature, AHU_1's fan run hours, and the number of readings where duct static was above 1.5 inH2O. (Hint: you can't do it cleanly yet. Try, notice what makes it awkward, and keep the attempt for Day 9.)

As a warm-up, do it in pandas first: three `groupby("month")` calls on three loaded series, then `pd.concat(..., axis=1)`. That is your oracle. Once Day 9's SQL gives a one-query answer, `lab.assert_parity` the two. Notice what is easy in pandas (separate frames, glued by the index) and awkward in SQL (one table, one `WHERE`, but three series).

Reading — Week 1

The lessons first, then these for exact behavior.

- Python sqlite3 module
- pandas: comparison with SQL
- DB Browser for SQLite
- SQLite SELECT
- SQLite date and time functions
- SQLite aggregate functions
- RFC 9562: UUIDs (version 5 = name-based)